

Task 1: Founder Doc Signal Machine

What it is, how it works, what the evidence means, and how n8n fits

One-sentence explanation. Task 1 is an offline, draft-only growth system that turns a human-researched CSV of small B2B founder prospects into claim-checked, personalized Markdown outreach drafts and a reproducible evidence bundle. It intentionally never sends anything.

16	16	0	2
fresh synthetic prospects	drafts generated	emails sent	recorded runs

THE MOST IMPORTANT DISTINCTION

This is not an autonomous lead scraper and it is not an email campaign. Humans find and research candidate companies. Automation begins only after a structured CSV is prepared. The system validates that input, maps each prospect's document pain to a SuperDocs angle, drafts a message, checks claims, and records evidence.

Added for n8n users: an importable workflow under n8n/ acts as a visual control plane around the existing Python implementation. The original implementation remains the single source of truth.

10 AUGUST 2026

1. Why Task 1 exists

The assignment calls for a growth machine with four concrete parts: a defensible audience, a runnable system executed twice, an audience-facing piece, and honest measurement. Task 1 maps to that requirement directly.

Requirement	Where it appears
Audience	AUDIENCE.md defines one narrow behavioral group and a method for finding initial companies.
Runnable system	machine/ contains the Python pipeline; data/ contains two input batches.
Two executions	runs/run-1/, runs/run-2/, and matching outbox folders preserve evidence.
Audience-facing piece	piece/index.html turns the hypothesis into a focused landing page.
Measurement	MEASUREMENT.md separates operational evidence from unavailable campaign outcomes.

The selected audience is **seed-stage technical founders and founding engineers at roughly 2-25 person B2B product companies** who already use coding agents but still finish proposals, investor updates, questionnaires, policies, or one-pagers by hand.

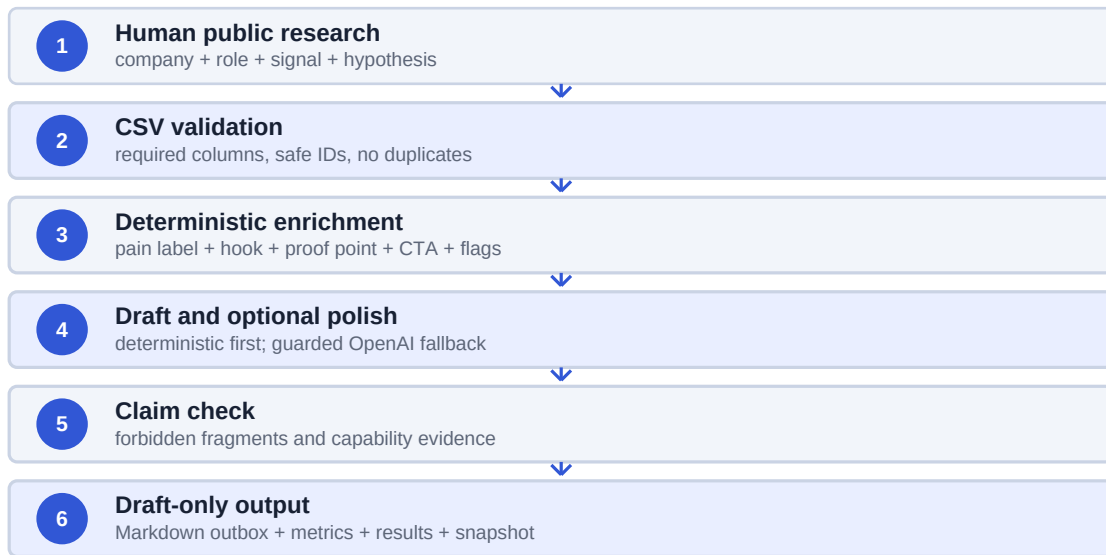
The insight is simple: **chat may solve the words while the document handoff remains manual**. SuperDocs is positioned as closing that last mile inside the actual document, with review, history, and style-preserving export.

What is deliberately out of scope

- No outreach to real people or companies during this round.
- No private contact database and no invented personal names.
- No web scraping inside the automation.
- No product claims beyond capabilities documented as available today.
- No conversion, reply, or revenue claims from unsent drafts.

2. The complete pipeline

The automation boundary starts at the CSV. Finding and reading public sources is a human research stage.



The system is therefore best described as a **researched-data-to-draft production machine**, not a prospecting robot.

Why deterministic generation is the default

The recorded runs use `--no-llm`. Given the same code and batch, the message structure and wording are reproducible. This makes review, debugging, and evidence comparison easier. Optional OpenAI polishing is an enhancement with a strict fallback, not a dependency.

The output boundary

The final automated action is writing files. There is no email provider, CRM sender, webhook-to-outreach platform, or hidden delivery step. The generated messages are artifacts for review only.

3. Folder-by-folder explanation

Strategy and operating documents

Path	Role
README.md	Short operating entry point with commands and evidence pointers.
AUDIENCE.md	Defines the audience, current failure mode, switch criteria, finding method, and explicit cuts.
MEASUREMENT.md	Records the two runs, explains the claim-check correction, and states the next valid experiment.
data/	Two CSV batches of eight synthetic company-role records each.
machine/	Typed ingestion, deterministic enrichment, personalization, claim checking, and orchestration.
outbox/	Draft-only Markdown messages, grouped by run ID.
runs/	Metrics, per-row results, exact batch snapshots, and readable summaries.
piece/index.html	Audience-facing landing page for the document-handoff wedge.
tests/	Regression checks for claim scope and path-safe IDs.
n8n/	New visual orchestration wrapper and setup guide.
docs/	This explanatory PDF and its LaTeX source.

The input record

Field	Meaning
prospect_id	Stable, path-safe identifier used in outputs.
company + role	Targets a seat at a company without a private person's name.
public_signal + public_url	Public basis for the research hypothesis.
product_one_liner	Brief description of the prospect's own product.
doc_pain	Category used to select the personalization playbook.
recent_public_detail	Specific context for a non-generic opening.
hypothesis	The document workflow pain being tested.
is_synthetic	Supports evidence labeling and risk flags.

4. What each Python module does

Module	Responsibility	Notable behavior
config.py	Product and filesystem configuration	Hard-codes SEND_MODE = False; defines allowed and forbidden claim language.
prospects.py	CSV ingestion	Requires columns; rejects duplicate and unsafe IDs; stores role/company rather than names.
enrich.py	Rule-based research enrichment	Maps pain to label, hook, proof point, CTA; adds soft risk flags.
personalize.py	Draft generation and claim checking	Builds deterministic draft first; optional OpenAI polish must pass length and claim gates.
run.py	Batch orchestration	Writes outbox drafts, metrics, detailed results, CSV snapshot, and summary.
tests/test_machine.py	Regression protection	Covers claim scope, forbidden claims, capability requirement, and path traversal.

Pain playbook

Document pain is normalized into one of five practical categories: investor update, customer proposal, security questionnaire, launch one-pager, or policy pack. Unknown values safely fall back to customer proposal. Each category supplies a concrete pain label, hook angle, proof point, and restrained CTA.

Optional LLM behavior

- 1 Always create a deterministic base draft first.
- 2 Only call OpenAI if LLM use is enabled and an API key exists.
- 3 Request JSON with subject and body under strict messaging rules.
- 4 Reject malformed, overly long, unsafe, or capability-free output.
- 5 Fall back silently to the deterministic draft if anything fails.

5. Safety and quality controls

Control	Purpose
Hard no-send switch	run.py refuses to execute if SEND_MODE is true. No sender exists.
Safe identifiers	Run IDs and prospect IDs cannot contain traversal paths or unsafe characters.
Company + role only	Avoids storing or fabricating private names.
Forbidden claim fragments	Rejects wording such as SOC2 compliance, web browsing, e-signature, and guaranteed redaction.
Concrete capability requirement	A message naming SuperDocs must include a supported capability marker.
Product-scoped checking	Prospect context is separated from the product claim section to avoid false positives.
Deterministic fallback	Network or model failure cannot stop draft production or silently bypass checks.
Exact batch snapshot	Every run preserves the input that produced it.
Regression tests	The discovered SOC2-context bug and path safety rules stay protected.

A useful example of iteration

An early recording found that public SOC2 context about a prospect was being treated as a SuperDocs claim. The checker was narrowed to the message subject and the product-related text, and regression tests were added. This is meaningful machine evidence: a false positive was observed, localized, fixed, and locked into tests.

Important limitation

The checker is rule-based. Zero failures means no current rule was violated. It does not prove that every possible unsupported implication has been detected, so human review remains necessary before any future send.

6. What the recorded results mean

Metric	Run 1	Run 2	Combined
Fresh prospects	8	8	16
Drafts written	8	8	16
Completion rate	100%	100%	100%
Claim-check failures	0	0	0
Rows with risk flags	0	0	0
Emails sent	0	0	0
Batch overlap	n/a	0	0

What is established: two fresh synthetic batches were accepted; every row produced a draft; the current claim checker passed every draft; no research risk flags were recorded; the two batches did not overlap; and no emails were sent.

What is not established: message-market fit, open rate, reply rate, qualified meetings, pipeline, or revenue. Because outreach is explicitly forbidden, any such figure would be fabricated.

How to interpret 100% completion

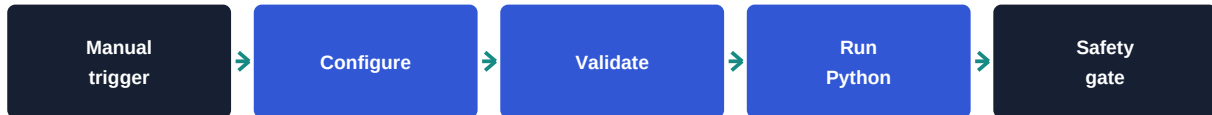
It is an operational measure showing that the production pipeline completed its known synthetic inputs. It is not a quality score from recipients. The proposed future quality rubric requires human scoring on signal specificity, pain plausibility, product fit, and CTA restraint before any permitted send.

Next experiment

If outreach later becomes permitted, manually review five drafts and test one framing variable: document-loop framing versus workflow-specific framing. The primary outcome would be qualified replies, with claim accuracy and negative response rate as guardrails.

7. The n8n workflow

The new workflow is a visual control plane around the tested Python machine. It does not duplicate the prospect, enrichment, personalization, or claim-check logic.



Node behavior

- 1 **Manual Trigger:** a person must intentionally start the workflow.
- 2 **Workflow Configuration:** choose the mounted project directory, CSV batch, safe run ID, and LLM setting.
- 3 **Validate and Build Command:** reject traversal, unsafe IDs, and malformed paths before shell execution.
- 4 **Run Existing Python Machine:** execute `python3 -m machine.run` in the mounted folder.
- 5 **Safety Gate and Run Summary:** parse metrics and fail unless send mode is false, emails sent is zero, all rows drafted, and claim-check failures are zero.

Why wrapper architecture is preferable

A native JavaScript rewrite inside n8n would create a second source of truth. The two versions could diverge in safety rules, wording, or evidence format. Wrapping the original machine gives an n8n-oriented operator visual configuration and execution history while preserving the tested behavior.

Runtime requirement

The workflow uses n8n's Execute Command node, so it requires self-hosted n8n and a read-write mount of this Task 1 folder. The node is not available in n8n Cloud and may be disabled by administrators.

8. How to run it

Original Python machine

Run these commands from `task-1-growth-machine/`:

```
python3 -m machine.run --batch data/batch-1.csv --run-id run-1 --no-llm
python3 -m machine.run --batch data/batch-2.csv --run-id run-2 --no-llm
python3 -m unittest discover -s tests -v
```

For a review run, use a fresh ID such as `review-run`. Reusing an existing run ID intentionally clears and regenerates the draft files in that run's outbox directory.

n8n import

- 1 Use self-hosted n8n with Python 3 available.
- 2 Mount `task-1-growth-machine` at an in-runtime path, for example `/files/task-1-growth-machine`.
- 3 Import `n8n/task-1-growth-machine.workflow.json`.
- 4 Update `projectDirectory` if the mount path differs.
- 5 Keep `useLlm=false` for the same deterministic behavior as the recorded runs.
- 6 Execute manually and inspect the final PASS summary plus the new `outbox/` and `runs/` folders.

Outputs from either route

Location	Contents
<code>outbox/</code>	One draft-only Markdown file per prospect.
<code>runs//metrics.json</code>	Aggregate production and safety metrics.
<code>runs//results.json</code>	Per-prospect claim, risk, generator, and file details.
<code>runs//batch.snapshot.csv</code>	Exact input used by this run.
<code>runs//SUMMARY.md</code>	Readable index of draft files.

9. Strengths, limits, and bottom line

Strengths

- The audience is behaviorally specific and tied to the actual product wedge.
- The machine is small, runnable, deterministic by default, and evidenced twice.
- Critical constraints are implemented in code rather than left as prose.
- The input snapshot and result files make every run auditable.
- Measurement clearly distinguishes operational completion from market impact.
- The n8n addition improves operability without creating a second implementation.

Current limits

- Research collection remains manual, and supplied company records are synthetic.
- Rule-based claim checking cannot detect every unsupported implication.
- A completed draft is not proof that a recipient would find it relevant or persuasive.
- Risk flags are soft checks and do not replace review of real public research.
- The n8n workflow requires self-hosting and filesystem access to the Python project.

Bottom line

Task 1 is a carefully bounded production prototype. Human public research enters as structured data. Deterministic automation produces personalized, claim-checked draft artifacts. Every execution leaves an audit trail. The system demonstrates reliable draft production while intentionally stopping before the prohibited act of contacting anyone.

Quick reference

Question	Answer
Does it scrape leads?	No. Human research is completed before the CSV stage.
Does it send email?	No. Sending is hard-disabled and absent from Python and n8n.
Does it need an LLM?	No. The recorded runs are deterministic.
What does it create?	Markdown drafts plus metrics, detailed results, an input snapshot, and a summary.
What proves it ran?	Two runs/ evidence folders and matching outbox/ drafts.
What is n8n doing?	Configuring, running, and gating the existing Python machine visually.
What remains unproven?	Real-world response, conversion, and market impact.